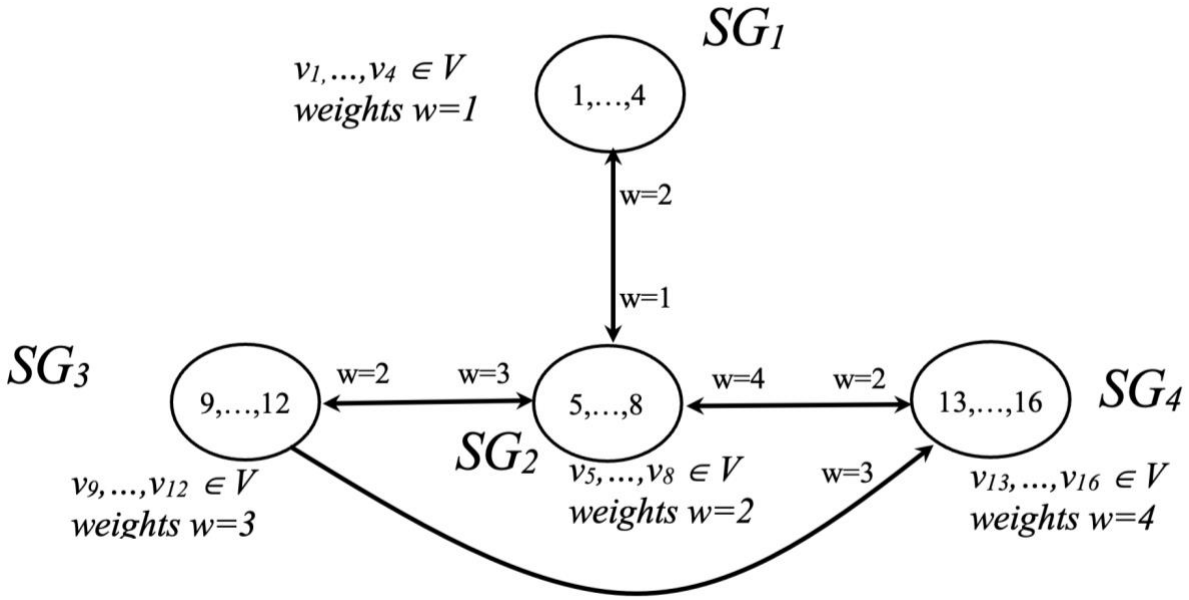
Q1.

Consider the digraph with 4 subgraphs SG_i, $i=1,\ldots,4$ below! The set of vertices $V=\{v_1,\ldots, v_{16}\}$, $|V|=16$, with 4 vertices at each subgraph as in the figure. Each subgraph is ***complete or fully connected***. Each neighbor subgraph is connected as shown in the figure. The bidirectional connections between subgraphs that symbolize two connections, one in each direction, are always *from the highest indexed vertex of the source subgraph SG_i to lowest indexed vertex of the destination subgraph SG_j* (**e.g.**, the edge from SG_3 to SG_2 is $e=(v_{12},v_5)$ and the edge from SG_2 to SG_3 is $e=(v_8,v_9)$). The edges are weighted such that each subgraph SG_i, $i=1,\ldots,4$ has a weight of $w=i$ for each edge *within* the subgraph and that interconnects other subgraphs (for example: each edge that connects any vertex pair within SG_1 and any edge going out of v_4 has a weight of w=1 while each edge within SG_3 and any edge going out of v_9 has a weight of w=3).

a) Find the ***shortest path*** **to each vertex in the graph from v_1** and **draw a graph that shows *only* these paths with the weights for each edge on that graph.**
b) What is the **total weight from v_1 to v_{16}**?

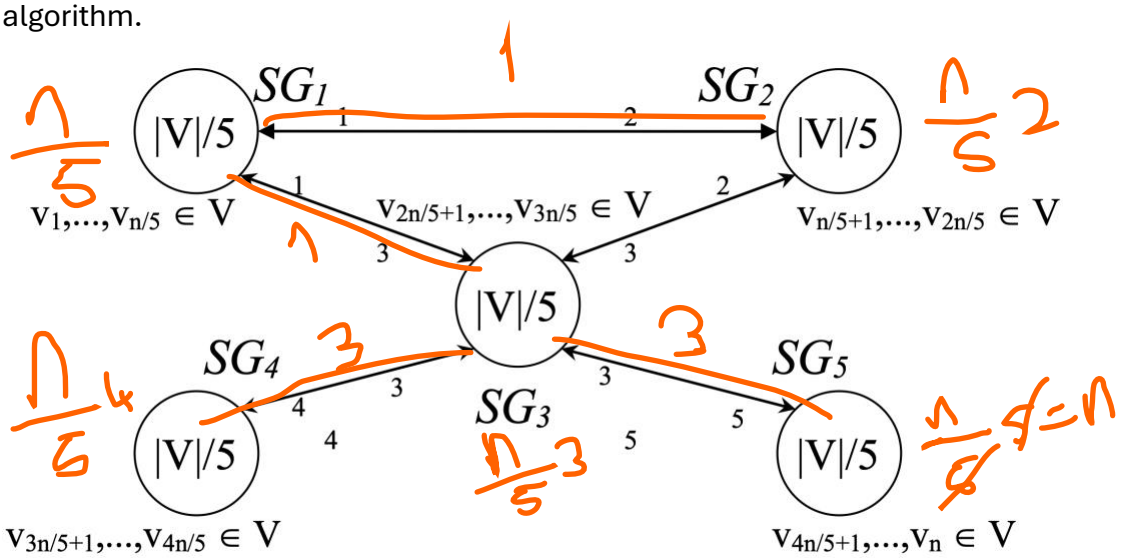
Hint: You may use the relevant algorithm from our in-class discussions to solve this problem; but it is easier to figure out the graph and solve it directly without using the algorithm.



Q2.

Consider the digraph below! Each subgraph SG_i has 1/5 of all vertices in the set of vertices V. The distribution of vertices to each subgraph is shown in the figure. Each subgraph is ***internally fully connected***. Each neighbor subgraph is connected as shown in the figure. The connections between subgraphs are always from the highest indexed vertex $u^i_{i_{max}}$ of the source subgraph SG_i to lowest indexed vertex $v^j_{j_{min}}$ of the destination subgraph SG_j (for instance, the edge from SG_3 to SG_2 goes from $v_{3n/5}$ to $v_{n/5+1}$). The edges are weighted such that each subgraph SG_i, $i=1,\ldots,5$ has a weight of i for each edge *within* the subgraph and interconnects other subgraphs (for example: each edge that connects any vertex pair within SG_1 and any edge going out of $v_{n/5}$ has a weight of 1 while each edge within SG_3 and any edge going out of $v_{3n/5}$ has a weight of 3).

Hint: You may use the relevant algorithm from our in-class discussions to solve this problem; but it is easier to figure out the graph and solve it directly without using the algorithm.



$8 + \frac{n}{5}(1+2+3+4+5) = 3n+8$

a) Find one of the minimum spanning trees MST_i of the above graph that ***minimizes the average distance*** p_i^{ave} ***between all pairs of vertices***?

$$p^i_{ave} = \frac{1}{|V|^2}\sum_{u\in V}\sum_{v\in V} p_i(u,v),$$

where $p_i(u,v)$ is the shortest path between vertices u and v in MST_i.

b) The following functions operate very similarly as the depth first search. In fact, the only difference is the printf functions in fcn2. Apply them to the graph above! Disregard the weights of edges for this part of question! Perform your search regarding the vertices' ascending numerical order! What is the final output of the algorithm at the end of execution for the above graph?

```
fcn1(G) {
   initialize all vertices;
   for each vertex u∈V[G] {
      status[u]=undetected;
   }
   for each vertex u∈V[G]
      if (status[u]==undetected)
            fcn2(u);
}
```

```
fcn2(u) {
     status[u]=detected;
     time++;
     d[u]=time;
     printf("V%d:%d ",ID[u],d[u]);

     for each v ∈ Adj[u]
          if (status[v] == undetected) {
            fcn2(v);
          }
     status[u]=processed;
     f[u]=++time;
     printf("V%d:%d ",ID[u],f[u]);
}
```

CSE2225 - Questions From Previous Final Exams

Q3.

Consider a **maximum binomial heap** BH_0 with $n=2^k - 3$ nodes. **Justify all your answers**!

a) What is the **exact** number of nodes, **in terms of *n***, the ***maximum key*** may be located at? k-1. How can we rewrite the result in terms of n?

b) Is it possible for the ***minimum key*** to be at a **root in BH_0?** Yes, B0

c) If the ***minimum*** key were known to be in the ***most crowded depth level of the second most crowded binomial tree in BH_0***, what would be the **exact** number of nodes, **in terms of *k***, you need to check to find it?

$\frac{1}{2^{k-4}!}$

d) BH_0 is ***merged*** with BH_1 (another maximum binomial heap) with $2^m + 1$ nodes (m>>k) after ***3 deletemin*** operations at BH_1. The resulting heap of the merge operation is BH_{res}.

1. What binomial trees is BH_{res} composed of?
2. How many nodes does BH_{res} have in terms of *k* and *m*? 2^m + 2^k - 5
3. Suppose now that the ***minimum*** key is at the most crowded binomial tree of BH_{res}. What would be the **exact** number of nodes you need to check to find it? 2^(m-1)

Q4.

Consider two binomial queues BQ_1 and BQ_2 with $n=2^k$ and $m=2^l$ elements, respectively, where

$k<l$.

a) One *DeleteMin* is performed at each binomial queue and then they are merged to BQ_M. How long does it take, in average, to find the minimum key in BQ_M?

O(k) or O(log n)

b) Suppose that BQ_1 and BQ_2 are first merged, and then two *DeleteMin*s are performed. Suppose further that both minimum keys are at B_k. The second minimum key, located among the children of the root, is at the root of the most crowded binomial tree. What is your answer to the same question in part (a)?

O(

CSE2225 - Questions From Previous Final Exams

Q5.

Here is an array which has just been partitioned by the first step of quicksort:

16 9 21 3 17 18 28 34 30 41 53 64 54 86 48

a) Which of these elements could have been the pivot in this first step? If there are more than one possibility, show each possible element.

28 and 41

b) Considering your answer in (a), perform the second step of quicksort using Hoare's partitioning algorithm we discussed in class (*pivot is the first* among the relevant key sequence)! If there are more than one correct answer in (a) then **use the smallest one**.

Q6.

Here is an array which has just been partitioned by the first step of quicksort:

12 13 7 16 4 18 26 23 25 32 41 54 35 65 33

a) Which of these elements could have been the pivot in this first step? If there are more than one possibility, show each possible element.

18 and 32

b) Considering your answer in (a), perform the second step of quicksort using Hoare's partitioning algorithm we discussed in class (*pivot is the first* among the relevant key sequence)! If there is more than one correct answer in (a) then use the largest one.

Q7.

Insert the keys ***219, 375, 89*** into the hash table of length ***m=13*** using open addressing with the hash function $h_1(k)=k \bmod m$. Use ***double hashing*** ($h_2(k)=R-(k \bmod R)$) as the collision resolution strategy in $h_1'(k,i) = [h_1(k) + i * h_2(k)] \bmod m$ where *R* is determined by following our in-class discussions. Justify all your hashing procedure!

	0	1	2	3	4	5	6	7	8	9	10	11	12
keys													
#collisions													

CSE2225 - Questions From Previous Final Exams

Q8.

a) Insert the keys ***211, 376, 259, 127, 69, 84, 261, 273, 217*** into the hash table of length ***m=13*** using open addressing with the hash function $h_1(k)=k \bmod m$. Use ***double hashing*** as the collision resolution strategy with $h_2(k)=R-(k \bmod R)$ where R is determined by following our in-class discussion; calculate the collision frequencies (CF) into the table!

CF	After\Hashtable	0	1	2	3	4	5	6	7	8	9	10	11	12
	Empty													
0	211				211									
0	376													376
1	259					259								
9	X=													

b) Find a key, X, in the last line of the table above (i.e., after all keys are in part 1.a are placed in the hash table) that can be placed in the hash table only after ***9*** collisions.

Empty: 2,5,8,11. Difference 3. h1(X)=1, h2(X)=3

c) (4+4 pts)

1. Find the average costs for insert and successful search for this table after all keys(including X) have been placed!
2. When would you decide to rehash? What would be the new table size (m_{new}) and R in the second hash function $h_2(k)$.